

CauldronHook

Uniswap v4 Hook Routing Allowlist — Reviewer Brief

Magic Internet Frens · mifrens.xyz

Why this submission exists

CauldronHook declares `beforeSwapReturnDelta` and `afterSwapReturnDelta`. Both are structurally required by the hook's single economic mechanism — an **ETH-denominated** swap fee — for reasons that follow from v4's specified/unspecified currency semantics rather than from a design preference. Section 2 shows the derivation.

We do **not** deploy at `0x91...`, and we do **not** target major pairs: every pool this hook serves is ETH / <our own iteration token>, created by our own registry.

Answers to the stated criteria

Upgradeable?	No — no proxy, no delegatecall in the hook, immutable bytecode (§4)
Requires custom data inputs?	No — <code>hookData</code> is optional on every path; empty is a first-class case (§5)
Audited?	Yes — independent from-scratch review, 340 tests incl. live-v4 fork tests (§6)
Differentiated?	Yes — §1
Deployment address	<code>0x31ee...50cc</code> — not <code>0x91</code>
Pairs	ETH / own token only — never a major pair

1 What the hook is

The Cauldron is an autonomous, self-funding token machine. One *iteration* token trades at a time; when its rolling 24-hour volume falls below a threshold, governance selects the next one, protocol-owned liquidity is recovered, and holders migrate 1:1. The cycle is designed to run indefinitely, and the protocol is funded entirely by its own swap fees.

A break-glass custody path does exist and we would rather state it than have it found: `emergencyWithdrawLP` can remove a generation's protocol-owned liquidity to an emergency admin. That admin is **immutable** (fixed at construction, no setter), the action must be announced on-chain and is delayed, an independent guardian can veto it in the window, and arming it *forces the redemption exit open* so holders can leave at the floor before anything moves. It reaches protocol-owned liquidity only — nothing in the system can mint, freeze, or move a token or NFT held in a user's wallet, and none of it is reachable from the hook. Full detail in §4.

Four things are genuinely v4-native rather than ported from a v2/v3 design:

- **The fee engine is the hook.** There is no LP fee (`POOL_FEE = 0`); 100% of trading revenue is the hook's ETH-denominated return-delta fee, routed per-swap to an NFT-holder dividend, a per-collection token floor, the iteration's proposer, and the reserve that funds the next launch.
- **Hook-native perpetuals.** Leveraged longs and shorts whose opens, closes and liquidations are *real pool swaps*, with no external oracle. Liquidation marks come from the engine's own on-chain TWAP ring. Underwater positions are swept opportunistically from `afterSwap`, so any swap on any interface can trigger a liquidation and earn the keeper reward.
- **Progressive in-swap seeding.** Launch liquidity streams into the pool over a window using `core modifyLiquidity` from inside `afterSwap` — keeperless, and un-frontrunnable because the deployed fraction is a pure function of elapsed time. Anti-snipe by book *shape* rather than by tax alone.

- **Commit–reveal NFT forging in afterSwap.** A direct Uniswap or aggregator swap — with no router and no hookData — forges NFTs in-swap. This is a deliberate first-class path, not a fallback (§5).

2 Permissions, and why the delta flags are required

2.1 Declared permissions

```
afterInitialize: true // adopt only pools our registry created
beforeSwap: true // charge the ETH fee on buys
afterSwap: true // charge the ETH fee on sells
beforeSwapReturnDelta: true
afterSwapReturnDelta: true
// everything else false: no initialize/donate/liquidity callbacks
```

Encoded flag bits: `0x10cc = 01000011001100`. The deployed address `0x31ee3436d96533e91874cda5dfc70cb521f750cc` carries exactly those low bits and no others — verifiable with `uint160(hook) & Hooks.ALL_HOOK_MASK`. We hold no liquidity-callback or donate permissions, so the hook cannot interpose on `modifyLiquidity` or `donate` at all.

2.2 The derivation

The hook’s economics require the fee to be denominated in **ETH on every trade**, because the token side of each pool is by construction a soon-to-be-retired asset — accruing protocol revenue in it would strand that revenue at the next rebirth.

In v4, `afterSwap`’s return delta can only act on the **unspecified** currency. For our pools (`currency0` is native ETH):

Quadrant	Specified	Unspecified	afterSwap could charge in
exact-in BUY (<code>zeroForOne, amt<0</code>)	ETH (in)	token (out)	token
exact-out BUY (<code>zeroForOne, amt>0</code>)	token (out)	ETH (in)	ETH
exact-in SELL (<code>!zeroForOne, amt<0</code>)	token (in)	ETH (out)	ETH
exact-out SELL (<code>!zeroForOne, amt>0</code>)	ETH (out)	token (in)	token

The **exact-input buy** — the dominant trade, and what the Uniswap interface and every aggregator emit — has the *token* as its unspecified leg. Charging it in ETH therefore requires skimming the specified input in `beforeSwap`, i.e. `beforeSwapReturnDelta`. There is no alternative mechanism.

`afterSwapReturnDelta` charges the sell side. Without it the hook cannot take value from a swap at all: a `poolManager.take()` with no offsetting return delta leaves an unsettled negative delta and the unlock reverts `CurrencyNotSettled`.

We evaluated the delta-free alternative and rejected it on merit

A dynamic LP fee (`DYNAMIC_FEE_FLAG` plus a `beforeSwap` override) would need no delta flag, and our LP is protocol-owned, so the fee would land in our own positions. We did not choose it, for reasons we think are the right ones rather than convenient ones:

1. **LP fees accrue in the input currency** (`Pool.sol: zeroForOne ? feeGrowthGlobal0 : feeGrowthGlobal1`), so every *sell* would accrue the retiring token — exactly the outcome the ETH-only fee exists to prevent.
2. **It would introduce a keeper.** Our positions sit in the periphery `PositionManager`, which opens its own unlock and therefore cannot be harvested inside a swap. Per-swap routing would become a batch harvest.
3. **It would be JIT-extractable.** We hold no liquidity-callback permissions, so `LPing`

into our pools is permissionless. Under an LP fee a searcher could JIT a concentrated position around a large swap and capture the fee — including the launch surtax, inverting the mechanic it exists for. Under the delta model this is structurally impossible, because the hook takes the fee directly regardless of who provides liquidity.

The delta flags are therefore load-bearing, and the alternative would make the hook *less* safe, not more.

3 Integration behaviour a router should know

We would rather you learn these from us than find them in review.

3.1 Disclosed: exact-output sells currently revert

`_beforeSwap` reverts `ExactOutSellUnsupported` for the (`amountSpecified > 0`, `!zeroForOne`) quadrant. The reason is the table in §2: the unspecified leg there is the token, so v4's return-delta mechanism physically cannot take an ETH fee. Rather than silently serve that quadrant *for free* — which was the prior behaviour, and a measured fee bypass — we refuse it.

Remediation, in progress

We accept that reverting a whole quadrant is a poor router citizen, and it is the one item in this brief we consider a genuine integration defect rather than a trade-off. Because the specified currency in that quadrant *is* ETH, the fee can be taken via `beforeSwapReturnDelta` on the specified side — which we already hold, so the fix requires **no additional permission** and no address re-mine. We are implementing it and will confirm before any mainnet routing goes live. Every other quadrant is served today.

3.2 Fee bounds

Component	Value	Notes
Base hook fee	300 bps (3%) default	Governance-tunable, hard-capped at <code>MAX_TAX_BPS = 1000</code> (10%)
Launch surtax	up to 9,600 bps	Time-decaying anti-snipe ; linear decay to 0 across <code>snipeWindowBlocks</code> (default 30 blocks). Routed 100% to NFT holders
Absolute clamp	9,900 bps	<code>MAX_TOTAL_FEE_BPS</code> — a swap always retains $\geq 1\%$ so it can never return zero
LP fee tier	0	Traders pay the hook fee only; there is no stacked LP fee

We flag the surtax explicitly because a hook that can take 99% of a swap deserves a second look. It exists only in the opening minutes of a freshly summoned pool, decays deterministically to zero, is bounded by an immutable `constant` ceiling, and is paid to token holders rather than to us. Steady state is 3%.

3.3 `afterSwap` does optional work — none of it can break a swap

`afterSwap` may trigger a nested buyback swap, a seeder liquidity placement, a perp-liquidation sweep, and an NFT mint. Every one of those is invoked with a **reserved gas budget and an ignored result**:

```

uint256 g = gasleft();
if (g > LIQ_GAS_RESERVE + LIQ_GAS_MIN) {
    perpEngine.call{gas: g - LIQ_GAS_RESERVE}(...); // result ignored
}

```

so an optional step that runs out of gas or reverts is a silent no-op, and the parent swap always completes its fee take and returns. Reserves are held for the liquidation sweep, the gacha step, the buyback, and the seeder poke independently. Nested swaps are guarded by transient flags (`_inSelfBuy`, `_inRelaunchClose`) so they never re-enter the fee logic or recurse.

3.4 Delta settlement

Four contracts may settle inside one unlock (hook, registry library, perp engine, seeder). Each settles its own deltas in its own frame. Two details the audit specifically verified:

- The buyback settles the **realised** debit read from the returned delta, not the intended amount — so a binding price limit cannot leave a positive un-taken delta and revert the user’s swap.
- The perp engine switches to an **in-place** swap when already inside the lock, rather than re-entering unlock.

3.5 Pool adoption is closed

v4 lets anyone initialise a pool naming any hook, so `_afterInitialize` is an untrusted entrypoint. We adopt a pool only if `sender == registry` and `currency0` is native ETH. A foreign pool naming this hook is *declined rather than reverted* — so we cannot be used to grief pool creation — and every value-moving path is gated on the adoption flag, meaning such a pool receives no fee logic, no accounting and no spending.

4 Not upgradeable

The hook is a plain contract. No proxy, no `delegatecall` in the hook, no admin that can replace its code. Three things could superficially read as upgradeability, so we describe them precisely:

1. **Policy modules** (`IDeathChecker`, `ISurtaxPolicy`, `IOddsPolicy`, `ICurvePolicy`, `IFeeRouter`). Each is a *view* that returns a number. It never custodies or moves funds, its output is clamped to the same immutable ceilings as the built-in rule, and a reverting module falls back to the built-in rule. `IFeeRouter` returns *amounts* only — the hook performs the sends, and a split that does not sum exactly to the fee is discarded in favour of the built-in split. This is how economic parameters are tuned without an upgradeable money contract.
2. **Controller pointer.** `setRegistry` is one-shot. Replacing it requires `proposeRegistryOverride` → a `REGISTRY_SWAP_DELAY` of **7 days**, declared *constant* so the owner cannot shorten their own notice period → `executeRegistryOverride`, which *flushes* the accrued fee reserve to the *outgoing* controller first, so a swap cannot capture ETH the previous controller accrued.
3. **Ownership.** The hook is owned by an `OpenZeppelin TimelockController` — an audited standard, chosen specifically to avoid an upgradeable proxy on the money path. Every parameter change is `schedule` → `wait` → `execute`.

4.1 The break-glass path, stated in full

Governance **can** tune fees and risk parameters, swap a policy module, pause a redemption flow, and — through an announced, delayed, guardian-vetoable action — **remove protocol-owned liquidity** via `emergencyWithdrawLP` / `emergencySweep` on the registry. We list this explicitly because it is a genuine custody power and any reviewer reading the source will find it.

Four things bound it:

- The emergency admin is **immutable** — set in the constructor with no setter, so it cannot be re-pointed at a fresh wallet after the fact. It is the governance timelock.
- Actions **must be announced on-chain** (`armEmergency`) before they can execute, and are then delayed. Our audit found that this was previously conditional on a non-zero delay — a zero-delay deployment skipped the arm, which silently disabled the forced-open exit below. It is now unconditional, so the guarantee holds at any configuration (audit finding F-19, fixed).
- An independent **guardian can veto** during that window. The guardian can only ever cancel — never propose, never move funds.
- **Arming forces the exit open.** The redemption circuit-breaker is overridden the instant an action is armed, so it cannot be used to trap holders ahead of a custody move.

Governance **cannot** mint tokens, freeze balances, pause transfers, touch any token or NFT held in a user's wallet, or shorten its own announced delays. **None of this is reachable from the hook** — these are registry functions over protocol-owned liquidity, and they cannot affect swap execution, settlement, or any routed trade.

5 No required custom data inputs

`hookData` is **optional on every code path**, and an empty `hookData` is a designed-for case rather than a degraded one:

- **Fee tier** — `_taxedPlayer` falls back to `sender` when `hookData` is absent.
- **NFT credit** — falls back to `tx.origin`, precisely so that a buyer arriving through the Uniswap interface or an aggregator still earns credit rather than having it credited to the router.
- **Fee exemption** — requires `isOpener[sender]`, an explicit allowlist. A direct swapper controls `hookData` freely, so an arbitrary caller cannot tag an exempt address and dodge the fee.

The in-swap NFT forging path exists *specifically* for router-less swaps. A swap with empty `hookData` is fully served: it is charged correctly, it accrues credit, it can forge an NFT, and it can trigger a perp liquidation.

6 Audit and testing

- **Independent audit** — a from-scratch review of all ~9,200 lines, conducted against the intended L2 deployment target, deriving its findings solely from source. Companion document: `MagicFrens_Independent_Audit_2026-09.pdf`. No critical findings; every Medium and Low finding either fixed with a regression test that fails against the pre-fix bytecode, or disclosed with remediation.
- **340 tests, 66 suites**, all passing: unit, pure fuzz, handler-driven invariants (128,000 calls per invariant), and **fork tests that drive live Uniswap v4** rather than a mock.
- **Verified structural properties**, argued from cited code in the audit's invariant register: fixed non-mintable supply; migration exactly 1:1 or it reverts; every v4 delta settled on all four settlement paths; pool adoption closed; and byte-identical storage layouts across the registry's delegatecall pair (53 entries, verified mechanically).

6.1 Disclosed limitations

We would rather state these than have them found:

Item	Status
Exact-output sells revert Reserve ceiling	Integration defect. Fix identified, needs no new permission (§3). Our redemption reserve is a single-sided band; if the token appreciates past it, redemption and migration pause until the next iteration. Protocol-internal — it does not affect swap routing or settlement.
blockhash-derived NFT rarity	Sound on Ethereum; weaker on our L2 target, where blockhash is documented as insecure. Being verified against the target chain before that feature is enabled there. Does not affect swaps.
EIP-170 headroom	The hook sits 88 bytes below the limit. A process constraint, not a user-facing risk.

7 One operational question

Our design summons a **new pool at every rebirth** — same hook, same ETH / <iteration token> shape, new `currency1` and therefore a new `PoolId`. The hook itself is permanent and immutable.

We have assumed allowlisting attaches to the **hook address**, which would make this submission a one-time review. If it instead attaches per-pool, we would appreciate knowing, since it materially affects how we operate — and we would rather align with your process now than discover the mismatch after launch.

8 Artifacts

Hook (current deployment)	0x31ee3436d96533e91874cda5dfc70cb521f750cc
Permission bits	0x10cc — afterInitialize, before/afterSwap, both swap deltas
Pair shape	ETH / <iteration token>, registry-created only
Independent audit	audit/MagicFrens_Independent_Audit_2026-09.pdf
Hook source	CauldronHook.sol (repo: contracts/solidity/)
Test suite	FOUNDRY_PROFILE=cauldron forge test
Public docs	mifrens.xyz/#!/docs (· machine-readable at /llms.txt)

In one paragraph

CauldronHook uses both swap delta flags because its only revenue mechanism is an ETH-denominated fee, and v4's unspecified-currency semantics make `beforeSwapReturnDelta` the sole way to charge an exact-input buy in ETH. It is not upgradeable, requires no custom data inputs, serves only pools its own registry created against native ETH, never touches a major pair, holds no liquidity or donate permissions, and every optional action it performs in `afterSwap` is gas-bounded and result-ignored so it cannot revert or starve a routed swap. It has been independently audited and is covered by 339 tests including live-v4 fork tests. The one known integration defect — a reverting exact-output-sell quadrant — is disclosed above with a fix that requires no additional permission, as is the registry's announced, delayed and guardian-vetoable break-glass over protocol-owned liquidity, which is not reachable from the hook and cannot affect a routed swap.